x86 FASM Assembly for Compiler Writers

A friendly, example-driven guide covering every topic used in the ICG compiler's code generation

What you will learn

This tutorial walks you through every concept your compiler uses when translating C source code to 32-bit x86 Linux assembly for FASM. Each topic starts with a plain-English explanation, then shows a minimal C snippet and the exact assembly your compiler would generate for it. No prior assembly knowledge is assumed.

Generated for the ICG Bison Compiler Project

0 Contents

1 FASM File Structure

Before your compiler writes a single instruction, it must write a *header* that tells FASM what kind of binary to produce. Think of it as the "cover page" of your assembly file.

1.1 The three mandatory lines

Listing 1: Minimal FASM ELF skeleton

```
format ELF executable 3    ; produce a 32-bit Linux ELF binary
entry main                 ; the OS will jump to the label "main"
   first

segment readable writeable ; data lives here (global variables)
  ; ... global variable declarations go here ...

segment readable executable ; code lives here
main:
  ; ... instructions go here ...
  MOV EAX, 1    ; sys_exit
  XOR EBX, EBX ; exit code 0
  INT 0x80
```

Note

format ELF executable 3 — tells FASM to produce a standalone executable (not a library). The `3` is the ELF ABI version.

entry main — without this, FASM wouldn't know where execution starts. `main` is just a label; it can be any name.

segment readable writeable — the data segment: the processor can read and write it, but not execute it.

segment readable executable — the code segment: the processor can read and execute it.

1.2 Corresponding C concept

Every C program starts at `main()`. In our compiler the global variable declarations are placed in the data segment and all function bodies go in the code segment.

2 Data Declarations (Global Variables)

Global variables live in the `readable writeable` segment. FASM uses `dd` (*define dword*) to reserve 4 bytes of space, and the `dup` operator to repeat an initial value.

2.1 Scalars and arrays

Listing 2: C global declarations

```
int x;          // scalar
int arr[5];     // array of 5 ints
```

Listing 3: FASM equivalents

```
segment readable writeable
  x   dd 1 dup (0)     ; 1 dword, initialised to 0
  arr dd 5 dup (0)     ; 5 dwords, each initialised to 0
```

Note

dd means "define dword" (4 bytes). Our compiler uses only `dd` because every `int` is 4 bytes wide in 32-bit mode.

N dup (0) is FASM syntax meaning "repeat the value `0` exactly `N` times". A scalar is just `1 dup (0)`.

Tip

Unlike MASM or NASM, FASM does *not* support the `times` directive when a label precedes it. Always use `dup` for named data.

3 Stack Frame / Function Prologue & Epilogue

Every function needs a small "setup" at the start and a matching "teardown" at the end. This is called the *stack frame*.

3.1 Why do we need a stack frame?

The **stack** is a region of memory that grows downward. **ESP** always points to the current top. **EBP** is used as a stable reference point (*base pointer*) so that local variables and parameters can always be found at fixed offsets even as ESP moves.

Listing 4: A simple function

```
int add(int a, int b) {
    return a + b;
}
```

Listing 5: Generated prologue and epilogue

```
; NOTE: the function is named "add_nums", not "add".
; "add" is a reserved mnemonic in FASM and cannot be used as a
   label.
add_nums:
    PUSH EBP            ; save caller's base pointer
    MOV  EBP, ESP       ; EBP now points to the top of the stack

    ; --- function body goes here ---
    MOV  EAX, [EBP+8]   ; load parameter a
    MOV  EBX, [EBP+12]  ; load parameter b
    ADD  EAX, EBX

add_nums_exit:
    ADD  ESP, 0         ; release local variables (none here)
    POP  EBP            ; restore caller's base pointer
```

```
    RET   8              ; return AND pop 8 bytes of parameters
```

Note

Reserved label names in FASM — FASM instruction mnemonics (`add`, `sub`, `mul`, `div`, `mov`, `push`, `pop`, `call`, `ret`, ...) cannot be used as label names because the assembler cannot tell them apart from instructions. Always give your function a name that does not clash with a mnemonic.

3.2 Stack layout picture

Address	Contents
EBP + 12	parameter `b`
EBP + 8	parameter `a`
EBP + 4	return address (pushed by CALL)
EBP + 0	saved EBP (pushed by us)
EBP - 4	first local variable
EBP - 8	second local variable

Note

RET 8 — the `8` tells the CPU to add 8 to ESP after returning, which pops the two 4-byte parameters. This is the *callee-cleanup* convention used by our compiler.

If the function has no parameters, use plain **RET**.

4 Local Variable & Parameter Layout

4.1 How offsets are assigned

The compiler keeps two counters:

- `nextLocalOffset` — starts at −4 and decreases by the variable size each time a new local is declared.
- `nextParamOffset` — starts at +8 and increases by 4 each time a parameter is registered.

Listing 6: Local variables and parameters

```
int foo(int x, int y) {
    int a;    // local 1
    int b;    // local 2
    a = x + y;
    b = a * 2;
    return b;
}
```

Listing 7: Offset assignments

```
; x   is at [EBP + 8]    (nextParamOffset started at 8)
; y   is at [EBP + 12]   (nextParamOffset incremented to 12)
; a   is at [EBP - 4]    (nextLocalOffset started at -4)
; b   is at [EBP - 8]    (nextLocalOffset decremented to -8)
```

```
5
6  foo:
7      PUSH EBP
8      MOV  EBP, ESP
9      SUB  ESP, 8         ; allocate space for a and b
10
11     MOV  EAX, [EBP+8]
12     MOV  EBX, [EBP+12]
13     ADD  EAX, EBX
14     MOV  [EBP-4], EAX ; store into a
15
16     MOV  EAX, [EBP-4]
17     MOV  EBX, 2
18     MUL  EBX
19     MOV  [EBP-8], EAX ; store into b
20
21     MOV  EAX, [EBP-8]
22
23 foo_exit:
24     ADD  ESP, 8         ; release a and b
25     POP  EBP
26     RET  8              ; pop x and y
```

Tip

The formula for local variable space to release is:

$$\texttt{ADD ESP,}\ ((-4) - \texttt{nextLocalOffset})$$

After declaring `a` and `b`, `nextLocalOffset` is -12, so: $(-4) - (-12) = 8$. That matches the `SUB ESP, 8` at the start.

5 Arithmetic Operations

5.1 Addition and subtraction

Listing 8: Addition and subtraction

```
1 int a = 10, b = 3;
2 int c = a + b;    // c = 13
3 int d = a - b;    // d = 7
```

Listing 9: ADD and SUB

```
1 MOV EAX, [EBP-4]   ; load a into EAX
2 MOV EBX, [EBP-8]   ; load b into EBX
3 ADD EAX, EBX       ; EAX = a + b  (result always ends up in EAX)
4
5 MOV EAX, [EBP-4]   ; load a again
6 MOV EBX, [EBP-8]   ; load b again
7 SUB EAX, EBX       ; EAX = a - b
```

5.2 Multiplication

Listing 10: Multiplication

```
int c = a * b;
```

Listing 11: MUL

```
MOV EAX, [EBP-8]   ; load b (right operand) first
PUSH EAX
MOV EAX, [EBP-4]   ; load a (left operand)
POP EBX
MUL EBX            ; EAX = EAX * EBX  (EDX gets the high 32 bits;
    ignore it)
```

Note

MUL EBX multiplies the *unsigned* 32-bit value in EAX by EBX and puts the lower 32 bits of the result back in EAX. The upper 32 bits go into EDX. Since our ints fit in 32 bits, we simply ignore EDX.

5.3 Division and modulo

Listing 12: Division and modulo

```
int q = a / b;    // quotient
int r = a % b;    // remainder
```

Listing 13: DIV for division and modulo

```
; -- division  (a / b) --
MOV EAX, [EBP-8]   ; load b
PUSH EAX
MOV EAX, [EBP-4]   ; load a  (dividend must be in EAX)
POP EBX
XOR EDX, EDX       ; clear EDX -- the DIV instruction reads EDX:EAX
DIV EBX            ; EAX = EAX / EBX,  EDX = EAX mod EBX
; result (quotient) is now in EAX

; -- modulo  (a % b) --
; same setup as above, then:
MOV EAX, EDX       ; move remainder into EAX so it becomes the
    expression value
```

Note

XOR EDX, EDX zeros EDX before `DIV`. The DIV instruction treats the pair `EDX:EAX` as a 64-bit dividend. If EDX is not zeroed you will get a division-overflow fault.

5.4 Unary negation

Listing 14: Unary minus

```
int b = -a;
```

Listing 15: NEG

```
MOV EAX, [EBP-4]   ; load a
NEG EAX             ; EAX = -EAX
```

6 Data Movement (MOV)

`MOV` is the workhorse of assembly. It copies a value from a source to a destination. The source is always on the *right*.

Listing 16: Forms of MOV used in our compiler

```
MOV EAX, 42           ; load the constant 42 into EAX
MOV EAX, [EBP-4]      ; load a 32-bit value from memory into EAX
MOV [EBP-4], EAX      ; store EAX into memory
MOV EBX, EAX          ; copy register to register
MOV byte [ESP], '-'   ; store the ASCII character '-' as a single
    byte
```

Note

Square brackets `[...]` mean "read from memory at this address". Without brackets the operand is a register or immediate value.

7 Stack Operations (PUSH / POP)

The stack is used heavily for two purposes: *saving registers* and *passing arguments* to functions.

7.1 Basic PUSH and POP

Listing 17: PUSH and POP

```
; PUSH decrements ESP by 4, then writes the value there.
; POP reads from [ESP], then increments ESP by 4.

PUSH EAX       ; save EAX on the stack
; ... do something that clobbers EAX ...
POP  EAX       ; restore EAX
```

7.2 Evaluating binary expressions

Our compiler evaluates the *right* operand first, pushes it, then evaluates the *left* operand, pops the right operand back into EBX, and performs the operation.

Listing 18: Saving an intermediate result

```
int result = a + b;    // left = a, right = b
```

Listing 19: Evaluate right, push, evaluate left, pop

```
MOV EAX, [EBP-8]    ; evaluate right operand (b)
PUSH EAX
MOV EAX, [EBP-4]    ; evaluate left operand (a)
POP EBX
ADD EAX, EBX         ; EAX = a + b
```

7.3 PUSH with a size qualifier

When pushing a value directly from memory, FASM requires a size qualifier so it knows how many bytes to push:

Listing 20: Pushing a memory value

```
PUSH dword [EBP-4]    ; push the 4-byte integer at [EBP-4]
```

8 Memory Addressing Modes

x86 supports several ways to specify a memory address. Our compiler uses five of them.

Mode	Example	Used for
Direct (label)	`[x]`	Global scalar
Base + displacement	`[EBP - 8]`	Local variable
Base + displacement (pos.)	`[EBP + 8]`	Parameter
Base label + index register	`[arr + EBX]`	Global array element
Base register + index reg.	`[EBP + ESI]`	Local array element

Listing 21: All five addressing modes

```
; global scalar x
MOV EAX, [x]

; local variable at offset -8 from EBP
MOV EAX, [EBP - 8]

; parameter at offset +8 from EBP
MOV EAX, [EBP + 8]

; global array:  EBX holds the byte offset (index * 4)
MOV EAX, [arr + EBX]

; local array:  ESI holds a negative offset so [EBP+ESI] == [EBP-N]
MOV EAX, [EBP + ESI]
```

8.1 Size qualifiers

When an instruction is ambiguous about operand size (because neither operand is a register), you must tell FASM:

Listing 22: Size qualifiers

```
INC dword [EBP-4]          ; increment the 4-byte integer at [EBP-4]
MOV byte  [ESP], 10        ; store the byte value 10 (newline) at [
    ESP]
PUSH dword [EBP-4]         ; push 4 bytes from memory
```

9 Array Indexing

An array in C is a contiguous block of integers. Because each `int` is 4 bytes, accessing element i means computing a *byte offset* of $i \times 4$ from the start of the array.

9.1 Global arrays

Listing 23: Global array access

```
int arr[10];

int main() {
    arr[3] = 99;
}
```

Listing 24: Index computation for a global array

```
segment readable writeable
  arr dd 10 dup (0)

segment readable executable
main:
    PUSH EBP
    MOV  EBP, ESP

    ; compute index expression (3) and multiply by element size (4)
    MOV EAX, 3        ; index = 3
    MOV EBX, 4
    MUL EBX           ; EAX = 3 * 4 = 12  (byte offset)
    MOV EBX, EAX      ; EBX = byte offset

    ; store the value 99 into arr[3]
    MOV EAX, 99
    MOV [arr + EBX], EAX    ; arr[3] = 99

main_exit:
    MOV EAX, 1
    XOR EBX, EBX
    INT 0x80
```

9.2 Local arrays

For local arrays the base address is at a negative offset from EBP. The compiler computes (index $\times$ 4) + |base_offset|, stores it in ESI, negates ESI, then uses `[EBP + ESI]` which is equivalent to `[EBP - offset]`.

Listing 25: Local array access

```
int foo() {
    int arr[5];       // local, at EBP-4 through EBP-20
    arr[2] = 7;
}
```

Listing 26: Index computation for a local array

```
; arr occupies [EBP-4]..[EBP-20]
; nextLocalOffset was -4 when arr was declared, so base_offset = 4

; step 1: compute index * 4
MOV EAX, 2        ; index
MOV EBX, 4
MUL EBX           ; EAX = 8

; step 2: add the base offset distance
MOV EBX, 4        ; |base_offset| = 4
ADD EAX, EBX      ; EAX = 8 + 4 = 12

; step 3: negate to get a negative displacement from EBP
MOV ESI, EAX
NEG ESI           ; ESI = -12  --> [EBP + (-12)] = [EBP - 12] = arr
   [2]

; step 4: use [EBP + ESI]
MOV EAX, 7
MOV [EBP + ESI], EAX
```

Note

The NEG+ESI trick avoids computing a new negative constant at compile time. Since ESI ends up negative, `[EBP + ESI]` is the same as `[EBP - N]`.

10 Comparison and Conditional Jumps

10.1 CMP and TEST

Listing 27: CMP and TEST

```
CMP EAX, EBX      ; computes EAX - EBX and sets CPU flags (ZF, SF, OF
   ...)
                  ; the result is thrown away; only the flags are
                     kept

TEST EAX, EAX     ; computes EAX AND EAX, sets ZF=1 if EAX==0
                  ; cheapest way to check "is EAX zero?"
```

10.2 Signed conditional jumps

Instruction	Meaning	C equivalent
`JE label`	jump if equal	`==`
`JNE label`	jump if not equal	`!=`
`JL label`	jump if less (signed)	`<`
`JLE label`	jump if less or equal	`<=`
`JG label`	jump if greater (signed)	`>`
`JGE label`	jump if greater or equal	`>=`
`JZ label`	jump if zero (same as JE)	
`JNZ label`	jump if not zero (same as JNE)	
`JMP label`	unconditional jump	`goto`

10.3 Producing a 0/1 boolean from a comparison

In C, a comparison like `a < b` evaluates to `1` (true) or `0` (false). Here is how the compiler produces that integer:

Listing 28: Relational expression

```
int result = (a < b);
```

Listing 29: Boolean result from CMP

```
; evaluate right operand first, push, then left operand
MOV EAX, [EBP-8]   ; b
PUSH EAX
MOV EAX, [EBP-4]   ; a
POP EBX

CMP EAX, EBX       ; compare a and b
JL  L_true         ; if a < b, jump to true branch
MOV EAX, 0         ; false
JMP L_end
L_true:
MOV EAX, 1         ; true
L_end:
```

11 Control Flow Patterns

Every C control structure maps to a pattern of labels and conditional jumps.

11.1 IF statement

Listing 30: IF statement

```
if (x > 0) {
    x = x - 1;
}
```

Listing 31: IF pattern

```
; evaluate condition
MOV EAX, [x]
MOV EBX, 0
CMP EAX, EBX
JLE L0                ; jump PAST body if condition is FALSE

; body
MOV EAX, [x]
MOV EBX, 1
SUB EAX, EBX
MOV [x], EAX

L0:                   ; execution continues here
```

11.2 IF-ELSE statement

Listing 32: IF-ELSE statement

```
if (x > 0) {
    x = 1;
} else {
    x = -1;
}
```

Listing 33: IF-ELSE pattern

```
MOV EAX, [x]
TEST EAX, EAX
JE L_else             ; if x == 0, go to else

; if-body
MOV EAX, 1
MOV [x], EAX
JMP L_end             ; skip the else body

L_else:
MOV EAX, 1
NEG EAX               ; EAX = -1
MOV [x], EAX

L_end:
```

11.3 WHILE loop

Listing 34: WHILE loop

```
while (x > 0) {
    x = x - 1;
}
```

Listing 35: WHILE pattern

```
L_start:
    MOV EAX, [x]
    TEST EAX, EAX
    JE  L_end          ; exit loop when x == 0

    MOV EAX, [x]
    MOV EBX, 1
    SUB EAX, EBX
    MOV [x], EAX

    JMP L_start

L_end:
```

11.4 FOR loop

Listing 36: FOR loop

```
for (i = 0; i < 5; i++) {
    x = x + i;
}
```

Listing 37: FOR pattern

```
; initialiser  (i = 0)
MOV EAX, 0
MOV [i], EAX

L_for_start:
; condition  (i < 5)
MOV EAX, 5
PUSH EAX
MOV EAX, [i]
POP EBX
CMP EAX, EBX
JGE L_for_end          ; exit when i >= 5

; body  (x = x + i)
MOV EAX, [i]
PUSH EAX
MOV EAX, [x]
POP EBX
ADD EAX, EBX
MOV [x], EAX

; increment  (i++)
PUSH dword [i]
INC dword [i]
POP EAX                ; (post-increment: result discarded here)

JMP L_for_start
```

```
L_for_end:
```

11.5 RETURN statement

A `return` is just a jump to the function's epilogue label:

Listing 38: RETURN

```
; return value is in EAX
MOV EAX, 42
JMP foo_exit    ; go directly to the epilogue
```

11.6 Label naming

The compiler generates unique labels `L0`, `L1`, `L2`, ... using a simple counter. Each function also has a `name_exit` label that the epilogue code lives under.

12 Logical Operators with Short-Circuit Evaluation

In C, `&&` and `||` are *short-circuit*: the right operand is not evaluated if the result is already known from the left.

12.1 Logical AND (`&&`)

Listing 39: Logical AND

```
if (a && b) { ... }
// if a is false, b is not evaluated
```

Listing 40: Short-circuit AND

```
; evaluate left operand
MOV EAX, [a]
TEST EAX, EAX
JE   L_skip           ; if a == 0 (false), skip to L_skip

; evaluate right operand (only reached if a != 0)
MOV EAX, [b]
JMP  L_next

L_skip:
MOV EAX, 0            ; result is 0 (false) without evaluating b

L_next:
; EAX now holds 0 or non-zero representing a&&b
TEST EAX, EAX
JE   L_end_if         ; if result is false, skip the body
; ... if-body ...
L_end_if:
```

12.2 Logical OR (||)

Listing 41: Logical OR

```
if (a || b) { ... }
// if a is true, b is not evaluated
```

Listing 42: Short-circuit OR

```
; evaluate left operand
MOV EAX, [a]
TEST EAX, EAX
JNE  L_skip           ; if a != 0 (true), skip to L_skip

; evaluate right operand
MOV EAX, [b]
JMP  L_next

L_skip:
MOV EAX, 1            ; result is 1 (true) without evaluating b

L_next:
```

13 Bitwise / Utility Instructions

Several bitwise instructions appear in the compiler not because the C source uses them directly, but because they are efficient tools for specific tasks.

Listing 43: Bitwise instructions used internally

```
XOR EDX, EDX    ; fastest way to zero a register (2 bytes, sets ZF)
                ; used before DIV to clear the high half of the
                   dividend

XOR ECX, ECX    ; same trick to zero a counter

OR  EAX, EAX    ; sets flags (including SF = sign flag) based on EAX
                ; does NOT change EAX; used to test sign of a number
                ; cheaper than CMP EAX, 0 for detecting negative
                   values
```

Tip

`XOR reg, reg` is preferred over `MOV reg, 0` for zeroing a register because it produces a shorter machine-code encoding and doesn't depend on the immediate operand.

14 Increment and Decrement (Post-fix)

C's post-fix `i++` and `i-` return the *old* value while updating the variable in memory.

Listing 44: Post-increment

```
int b = a++;    // b gets the OLD value of a; a is incremented
```

Listing 45: Post-increment sequence

```
; save the OLD value first
PUSH dword [EBP-4]    ; push current value of a

; now modify a in memory
INC dword [EBP-4]     ; a = a + 1

; retrieve the old value as the expression result
POP EAX               ; EAX = old value of a
```

Note

`INC`/`DEC` on a memory operand requires the `dword` size qualifier because FASM cannot infer the operand size from context.

15 Function Calls and Calling Convention

15.1 Calling a function

Listing 46: Calling a function with arguments

```
int result = add(3, 5);
```

Listing 47: Pushing arguments and CALL

```
; arguments are pushed RIGHT-TO-LEFT
; (last argument pushed first, so first argument is closest to EBP
   +8)
MOV EAX, 5          ; second argument
PUSH EAX
MOV EAX, 3          ; first argument
PUSH EAX
CALL add_nums       ; push return address, jump to add_nums
                    ; after CALL, EAX holds the return value
; (reminder: "add" is a FASM mnemonic; use a different name)
```

Note

CALL name does two things automatically: it pushes the address of the next instruction (return address) onto the stack, then jumps to `name`. **RET** pops that address and jumps back.

15.2 Preserving EAX across a CALL

If EAX holds an important value and you need to make a function call, save EAX first:

Listing 48: Saving EAX around a call

```
PUSH EAX            ; save our important value
; ... push arguments ...
CALL some_function
POP EAX             ; restore our value
```

15.3 Callee-cleanup convention

Our compiler uses **callee-cleanup**: the called function (*callee*) removes the arguments from the stack using `RET N` where $N = \text{number\_of\_params} \times 4$.

This means the *caller* does not need to add anything to ESP after CALL.

16 Linux System Calls (INT 0x80)

On 32-bit Linux, programs communicate with the OS through the `INT 0x80` software interrupt. Before executing the interrupt, you load specific registers with the system call number and its arguments.

Register	Holds
EAX	system call number
EBX	argument 1
ECX	argument 2
EDX	argument 3

16.1 sys_exit (terminate the program)

Listing 49: sys_exit

```
MOV EAX, 1          ; system call number for sys_exit
XOR EBX, EBX    ; exit code 0  (EBX = argument 1)
INT 0x80            ; call the kernel
```

16.2 sys_write (write bytes to a file descriptor)

Listing 50: sys_write

```
; ssize_t write(int fd, const void *buf, size_t count)
MOV EAX, 4          ; system call number for sys_write
MOV EBX, 1          ; file descriptor 1 = stdout
MOV ECX, msg        ; pointer to the buffer
MOV EDX, 1          ; number of bytes to write
INT 0x80
```

17 The OUTDEC Procedure (Printing Integers)

The C source uses `println(x)` to print an integer followed by a newline. The compiler emits a call to the helper procedure `OUTDEC` which converts an integer in EAX to ASCII digits and prints them via sys_write.

17.1 Overall algorithm

1. If EAX is negative, print `'-'` and negate EAX.
2. Repeatedly divide EAX by 10. The remainder is a digit (0–9). Convert it to ASCII by adding 0x30 (`'0'`). Push each digit onto the stack.
3. Pop and print each digit (this reverses the order, giving most-significant-digit first).

4. Print a newline (byte 10).

Listing 51: C equivalent of OUTDEC logic

```c
void println(int n) {
    if (n < 0) { putchar('-'); n = -n; }
    char digits[12];
    int count = 0;
    do {
        digits[count++] = '0' + (n % 10);
        n /= 10;
    } while (n != 0);
    for (int i = count-1; i >= 0; i--) putchar(digits[i]);
    putchar('\n');
}
```

Listing 52: Full OUTDEC procedure

```
OUTDEC:
    PUSH EBX
    PUSH ECX
    PUSH EDX
    PUSH ESI

    ; --- handle negative numbers ---
    OR  EAX, EAX          ; test sign
    JGE OUTDEC_POSITIVE
    NEG EAX               ; make it positive
    PUSH EAX              ; save positive value

    SUB ESP, 4            ; make room for one byte on the stack
    MOV byte [ESP], '-'   ; write '-' character
    MOV EAX, 4            ; sys_write
    MOV EBX, 1            ; stdout
    MOV ECX, ESP          ; buffer = [ESP]
    MOV EDX, 1            ; 1 byte
    INT 0x80
    ADD ESP, 4            ; free the byte

    POP EAX               ; restore positive value

OUTDEC_POSITIVE:
    XOR ECX, ECX          ; ECX = digit count
    MOV EBX, 10

OUTDEC_DIGIT_LOOP:
    XOR EDX, EDX          ; clear high half before DIV
    DIV EBX               ; EAX = EAX/10, EDX = EAX%10
    ADD DL, 30h           ; convert digit to ASCII
    PUSH EDX              ; push the character (as a dword, only DL
       matters)
    INC ECX               ; count it
    TEST EAX, EAX
```

```
    JNZ OUTDEC_DIGIT_LOOP    ; repeat while quotient != 0

    ; --- print digits in reverse (stack reverses the order) ---
    MOV ESI, ECX         ; save digit count
    MOV EBX, 1
    MOV EDX, 1

OUTDEC_PRINT_LOOP:
    TEST ESI, ESI
    JZ   OUTDEC_NEWLINE   ; printed all digits?

    MOV EAX, 4           ; sys_write
    MOV ECX, ESP         ; the top of stack holds the next digit byte
    INT 0x80
    ADD ESP, 4           ; pop the digit
    DEC ESI
    JMP OUTDEC_PRINT_LOOP

OUTDEC_NEWLINE:
    SUB ESP, 4
    MOV byte [ESP], 10   ; ASCII newline
    MOV EAX, 4
    MOV ECX, ESP
    INT 0x80
    ADD ESP, 4

    POP ESI
    POP EDX
    POP ECX
    POP EBX
    RET
```

Note

Callee-saved registers — OUTDEC saves EBX, ECX, EDX, and ESI at the start and restores them at the end. This is important because the *caller* may be using those registers and does not expect OUTDEC to change them. EAX is intentionally *not* saved because it is the input argument.

17.2 Calling OUTDEC from generated code

Listing 53: Using println

```
int main() {
    int x;
    x = 42;
    println(x);
}
```

Listing 54: println code generation

```
; save EAX (may hold something important)
```

```
PUSH EAX

; load the variable into EAX (OUTDEC reads from EAX)
MOV EAX, [EBP-4]    ; [EBP-4] is where x lives

; call the printer
CALL OUTDEC

; restore EAX
POP EAX
```

18 Register Roles and Usage Convention

Although x86 registers are general-purpose, our compiler assigns each one a consistent role.

Register	Role in our compiler	Notes
EAX	Primary accumulator; all results end up here	Also the return value
EBX	Second operand in binary ops; array offset	Clobbered freely
ECX	Loop counter (inside OUTDEC)	Clobbered freely
EDX	Remainder after DIV; high word of MUL	Clobbered by DIV/MUL
ESI	Negative displacement for local array access	Set during indexing
EBP	Frame pointer (base of current stack frame)	Never modified in body
ESP	Stack pointer	Adjusted for locals

Note

Because our compiler evaluates expressions left-to-right using EAX as the accumulator and PUSH/POP to preserve values, you do not need to worry about register allocation. Every intermediate result is flushed through the stack.

19 NOT Operator (Logical Negation)

C's ! operator turns any nonzero value into 0 and zero into 1.

Listing 55: Logical NOT

```
int b = !a;    // b = 1 if a==0, else b = 0
```

Listing 56: NOT pattern

```
MOV EAX, [EBP-4]        ; load a

TEST EAX, EAX           ; is a zero?
JNE  L_not_true         ; if a != 0, jump to "it was true"
MOV  EAX, 1             ; a was 0, so !a = 1
JMP  L_not_end

L_not_true:
MOV  EAX, 0             ; a was nonzero, so !a = 0
```

```
10
11 L_not_end:
```

20 Peephole Optimizer

After the main code generation pass, the compiler reads `code.asm` back and runs a simple *peephole optimizer* — a sliding two-line window that spots and eliminates wasteful patterns.

20.1 Pattern 1: PUSH/POP of the same register

Listing 57: Before optimization

```
1 PUSH EAX       ; save EAX
2 POP  EAX       ; restore EAX -- does nothing!
```

The optimizer detects that both instructions use the same operand and comments out *both*. This often happens when the right operand of a binary expression is also EAX.

20.2 Pattern 2: ADD or SUB 0

Listing 58: Before optimization

```
1 ADD ESP, 0     ; pointless; ESP is unchanged
```

Any `ADD` or `SUB` whose operand ends in the digit `0` (preceded by a space) is commented out.

20.3 Pattern 3: Redundant double MOV

Listing 59: Before optimization

```
1 MOV EAX, EBX   ; copy EBX -> EAX
2 MOV EBX, EAX   ; copy EAX -> EBX  -- EBX already had that value!
```

If `MOV A, B` is immediately followed by `MOV B, A`, the second instruction is redundant and gets commented out.

20.4 Pattern 4: Multiply by 1

Listing 60: Before optimization

```
1 MOV EBX, 1
2 MUL EBX        ; EAX * 1 = EAX, pointless
```

Both lines are commented out, leaving EAX unchanged.

Tip

A peephole optimizer does not require a full IR (intermediate representation). It works directly on the text of the assembly file, which is why it can be implemented as a simple string comparison loop.

21 Putting It All Together: A Complete Example

Here is a small C program that exercises almost every topic in this tutorial, followed by the corresponding FASM assembly your compiler would generate.

Listing 61: Full C example

```c
int g;            // global scalar

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

void main() {
    g = factorial(5);
    println(g);
}
```

Listing 62: Corresponding FASM assembly (hand-annotated)

```
format ELF executable 3
entry main

segment readable writeable
  g dd 1 dup (0)            ; global int g

segment readable executable

factorial:
    PUSH EBP
    MOV  EBP, ESP             ; prologue

    ; if (n <= 1) return 1
    MOV EAX, 1
    PUSH EAX
    MOV EAX, [EBP+8]          ; load n (parameter at EBP+8)
    POP EBX
    CMP EAX, EBX              ; compare n and 1
    JLE L0                    ; n<=1 is true?
    JMP L1
L0:
    MOV EAX, 1
    JMP factorial_exit        ; return 1

L1:
    ; return n * factorial(n-1)
    ; first compute n-1 and call factorial
    MOV EAX, 1
    PUSH EAX
    MOV EAX, [EBP+8]          ; n
    POP EBX
    SUB EAX, EBX              ; n - 1
```

```
33      PUSH EAX                    ; push argument (n-1)
34      CALL factorial              ; EAX = factorial(n-1)
35      ; now multiply n * EAX
36      PUSH EAX                    ; save factorial(n-1)
37      MOV  EAX, [EBP+8]           ; load n
38      POP  EBX                    ; EBX = factorial(n-1)
39      MUL  EBX                    ; EAX = n * factorial(n-1)
40      JMP factorial_exit
41
42  factorial_exit:
43      ADD ESP, 0                  ; no locals to release
44      POP EBP
45      RET 4                       ; pop 1 parameter (4 bytes)
46
47  main:
48      PUSH EBP
49      MOV  EBP, ESP
50
51      ; g = factorial(5)
52      MOV EAX, 5
53      PUSH EAX
54      CALL factorial               ; EAX = 120
55      MOV [g], EAX                ; store into g
56
57      ; println(g)
58      PUSH EAX
59      MOV  EAX, [g]
60      CALL OUTDEC
61      POP  EAX
62
63  main_exit:
64      MOV EAX, 1                  ; sys_exit
65      XOR EBX, EBX
66      INT 0x80
67
68      ADD ESP, 0
69      POP EBP
70      RET
71
72  OUTDEC:
73      ; ... (the full integer-printing procedure as shown in section
            16) ...
74      RET
```

22 Quick Reference Card

Instructions at a glance

Instruction	Effect
`MOV dst, src`	dst = src
`ADD dst, src`	dst = dst + src
`SUB dst, src`	dst = dst – src
`MUL src`	EAX = EAX * src (EDX = high bits)
`DIV src`	EAX = EDX:EAX / src, EDX = remainder
`NEG dst`	dst = –dst
`INC dst`	dst = dst + 1
`DEC dst`	dst = dst – 1
`PUSH src`	ESP -= 4; [ESP] = src
`POP dst`	dst = [ESP]; ESP += 4
`CMP a, b`	set flags for a–b (discard result)
`TEST a, b`	set flags for a AND b (discard result)
`JMP label`	unconditional jump
`JE / JNE`	jump if equal / not equal
`JL / JLE`	jump if less / less-or-equal (signed)
`JG / JGE`	jump if greater / greater-or-equal (signed)
`JZ / JNZ`	jump if zero / not zero
`CALL label`	push return addr, jump to label
`RET [N]`	pop return addr, jump back; ESP += N
`XOR r, r`	fastest way to zero register r
`OR r, r`	set flags without changing r
`INT 0x80`	Linux system call (number in EAX)

Stack frame cheat sheet

Address	Contents
EBP + 4N + 4	N*th* parameter (N = 1 ...)
EBP + 8	1st parameter
EBP + 4	return address
EBP + 0	saved EBP
EBP - 4	1st local variable
EBP - 8	2nd local variable
EBP - 4N	N*th* local variable

Linux int 0x80 system calls used

EAX	Call	Arguments
1	sys_exit	EBX = exit code
4	sys_write	EBX = fd (1=stdout), ECX = buf ptr, EDX = length

FASM data declarations

Syntax	Meaning
`name dd 1 dup (0)`	4-byte integer, initialised to 0
`name dd N dup (0)`	array of N ints, all zero
`db value`	define 1 byte
`dw value`	define 2 bytes (word)
`dd value`	define 4 bytes (dword)